funTDA — Part B: time series baseline (Tribolium beetle model)

Persistence landscapes, FPCA, and functional permutation tests over 400 simulated series

Victoria Arroniz

2026-07-30

Table of contents

Data

This is Part B of the funTDA project: the same persistence-landscape-based functional data analysis (FDA) pipeline used in Part A for gene regulatory networks, applied here to **simulated time series** instead. The system is the Tribolium flour beetle population model of Costantino et al. (1995). Pereira & de Mello (2015) is the reference for applying persistent homology to time series via Takens-delay-embedded point clouds, the general approach `beetles_landscapes.ipynb` follows here; this document has not independently verified that Pereira & de Mello's own worked examples use the Tribolium model specifically, as opposed to some other time series.

The discrete-time model tracks three life stages (larvae `L`, pupae/callow adults `P`, mature adults `A`) in two-week intervals. Two parameter regimes are simulated, 200 series each, 201 time steps, with initial conditions `L0, P0, A0 ~ U[2, 100]`:

- **Stable** (`u_a = 0.73`, adult death rate): the population converges to a fixed-point equilibrium.
- **Aperiodic** (`u_a = 0.96`): the population oscillates chaotically.

> **i** This functional test is new, not a reproduction
>
> Two things distinguish this document from a re-run of prior work:
>
> 1. **Higgins, Wu & Carey's funTDA paper does not analyze time series at all.** Its worked examples are all networks — Erdős–Rényi graphs, stochastic block models, a literary character network, and the H3N2 gene regulatory networks used in Part A. Applying the funTDA landscape + FPCA + permutation-test pipeline to time series is an extension of the method to a data modality the original paper never covers.
> 2. **The original `Example1Beetles.Rmd` does not run this functional hypothesis test either.** It runs two *independent* classification checks, each against the true regime labels, each reported as its own confusion matrix:
>
>   (a) k-means clustering of the raw (unnormalized) series into two groups, compared to the true labels (`cfm <- table(correct, km_clusters)`), and separately (b) a rule that calls a series "correctly" classified if its single most persistent bar has dimension 0 (stable) or 1 (aperiodic), again compared to the true labels (`cfm_hom`) — not to the k-means clusters from (a). The two checks are never compared to each other. There is no landscape, no FPCA, and no permutation test in it — both are classification-accuracy checks against ground truth, not a test of functional equality between group means.
>
> The FPCA and permutation test below are therefore a genuinely new analysis, not a reproduction of a previously reported number — unlike Part A, there is no prior "baseline" p-value or variance-explained figure to match. The `ref` values in this document's source are sanity checks from an earlier direct computation on these same committed arrays, not an external ground truth.

Pipeline: from simulated series to functional landscapes

The full pipeline — simulation, per-series min-max normalisation to `[0, 1]`, Takens delay embedding ($m = 2$, $\tau = 3$), Vietoris–Rips persistent homology via `ripser` (the same underlying algorithm `TDAstats::calculate_homology` uses in R), and the first H1 persistence landscape layer (`KK = 1`) on a shared 500-point grid — is implemented in Python, in `beetles_landscapes.ipynb`. The grid there is **data-adaptive**: `tseq = linspace(global_min_birth, global_max_death, 500)`, rather than the fixed `[0, 1]` Part A could use, because Rips filtration values on a normalised phase-space point cloud don't have a natural upper bound of 1.

That notebook saves three arrays (`stable_landscapes.npy`, shape `(200, 500)`; `aperiodic_landscapes.npy`, shape `(200, 500)`; `tseq.npy`, shape `(500,)`). `export_landscapes.py` converts them to the CSVs this document reads (`output/stable_landscapes.csv`, `output/aperiodic_landscapes.csv`, `output/tseq.csv`) — mirroring Part A's split between a Python stage that computes persistent homology and an R stage that does the functional data analysis. The landscapes are **not** re-simulated or re-computed here: doing so is slow (ripser over 400 point clouds), and Python's and R's random number generators aren't interchangeable, so re-simulating in R would not

reproduce these particular 400 series anyway. The committed CSVs are the single source of truth for everything that follows.

Reused figures from `beetles_landscapes.ipynb`

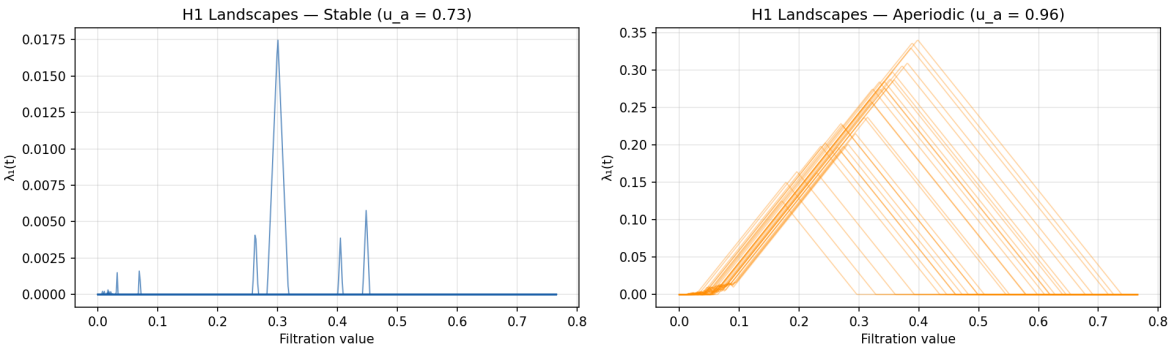

Figure 1: Individual H1 landscapes, 30 example series per regime (generated in the Python notebook, reused as-is).

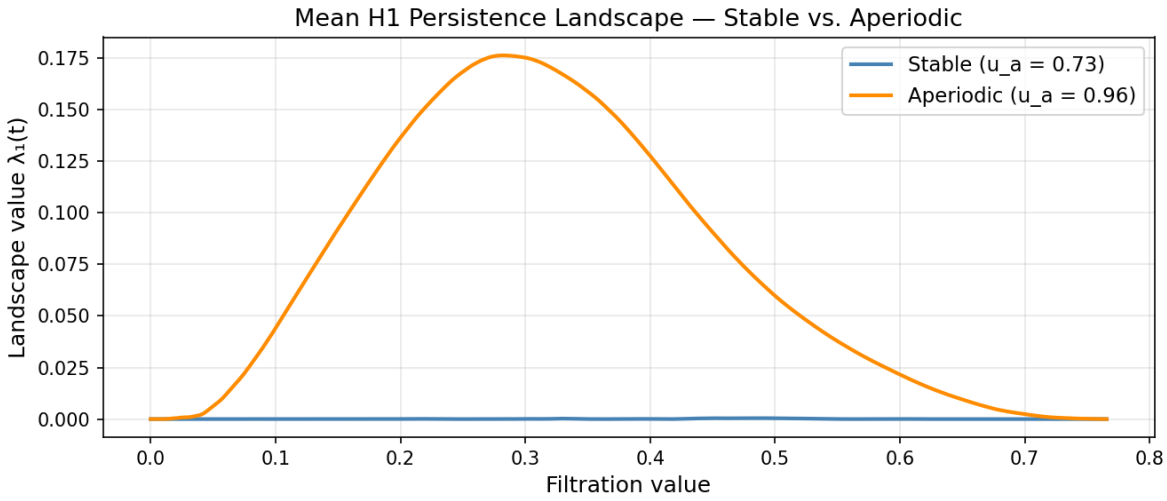

Figure 2: Mean H1 landscape per regime (generated in the Python notebook, reused as-is).

Structural check: most stable landscapes are identically zero

129 of the 200 stable series (64%) have a landscape that is identically zero across the whole grid, versus **0 of 200** aperiodic series. `u_a = 0.96` drives sustained oscillation, whose attractor is a closed orbit — an H1 loop that the landscape detects directly, so `n_aperiodic_zero = 0` reflects a genuinely empty diagram's absence. The stable case is more subtle and, on its own, easy to over-read: **these 129 series do not have empty H1 diagrams.** Every stable series still carries the decaying spiral of its approach to the `u_a = 0.73` fixed point, which is a closed curve in the Takens embedding and therefore does register an H1 feature — just one short-lived

enough that no point of the shared 500-point grid falls inside its birth–death interval, so the *sampled* landscape reads as zero even though the feature exists. "The approach-to-equilibrium transient" section below derives why this spiral exists and quantifies the gap between feature lifetime and grid spacing. Consistent with a real but small signal rather than a true absence, the mean of each series' landscape peak is **0.0016** for stable series versus **0.2274** for aperiodic ones — roughly a 150-fold difference in typical peak height, not a zero-vs- nonzero one.

Functional Principal Component Analysis (FPCA)

Identical smoothing to `funTDA.R`: an order-2 (piecewise-linear) B-spline basis with `nbasis = 499` (one fewer than the 500 grid points), so the basis nearly interpolates the raw landscape values. The only difference from Part A is the basis range: `range(tseq)` here (the data-adaptive Rips scale, about `[0, 0.77]`) instead of the fixed `[0, 1]` Part A could use. PC1's sign is fixed by convention (stable series score negative) immediately after `pca.fd`, since an eigenvector's sign is otherwise arbitrary — see the code below.

Table 1: Variance explained by the first two functional components

Component	% variance explained
PC1	84.29%
PC2	12.67%
Cumulative (PC1+PC2)	96.96%

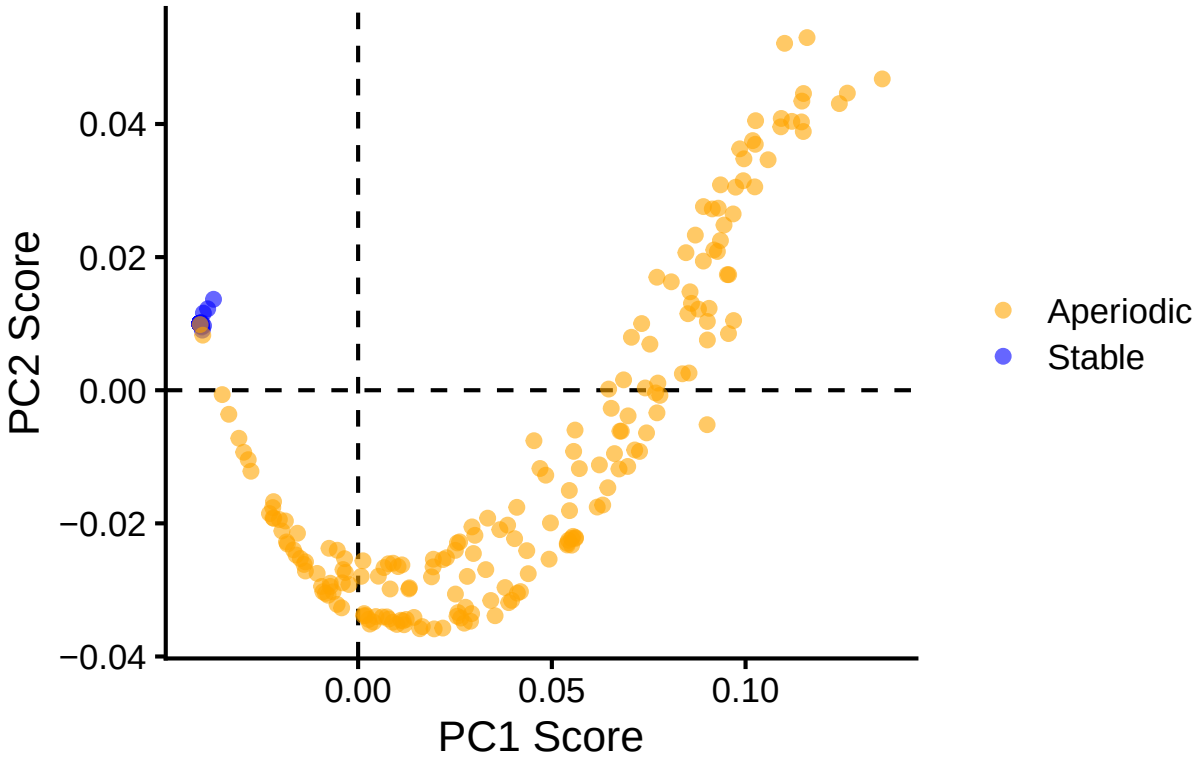

Figure 3: PC1 vs PC2 scores (FPCA over H1 landscapes), colored by regime. All 400 points are individual series; no per-point labels, unlike Part A's 17-subject plot, since labeling 400 points would be unreadable — see the full score table below for per-series values.

Full FPC scores (all 400 series)

ID	Group	PC1	PC2
Stable1	Stable	-0.0408	0.0100
Stable2	Stable	-0.0408	0.0100
Stable3	Stable	-0.0408	0.0100
Stable4	Stable	-0.0408	0.0100
Stable5	Stable	-0.0408	0.0100
Stable6	Stable	-0.0408	0.0100
Stable7	Stable	-0.0408	0.0100
Stable8	Stable	-0.0408	0.0100
Stable9	Stable	-0.0408	0.0100
Stable10	Stable	-0.0408	0.0100
Stable11	Stable	-0.0408	0.0100
Stable12	Stable	-0.0408	0.0100
Stable13	Stable	-0.0408	0.0100

Stable14	Stable	-0.0408	0.0100
Stable15	Stable	-0.0407	0.0101
Stable16	Stable	-0.0408	0.0100
Stable17	Stable	-0.0408	0.0100
Stable18	Stable	-0.0408	0.0100
Stable19	Stable	-0.0408	0.0100
Stable20	Stable	-0.0408	0.0100
Stable21	Stable	-0.0408	0.0100
Stable22	Stable	-0.0408	0.0100
Stable23	Stable	-0.0408	0.0100
Stable24	Stable	-0.0408	0.0100
Stable25	Stable	-0.0408	0.0100
Stable26	Stable	-0.0408	0.0100
Stable27	Stable	-0.0408	0.0100
Stable28	Stable	-0.0408	0.0100
Stable29	Stable	-0.0408	0.0100
Stable30	Stable	-0.0408	0.0100
Stable31	Stable	-0.0408	0.0100
Stable32	Stable	-0.0408	0.0100
Stable33	Stable	-0.0408	0.0101
Stable34	Stable	-0.0408	0.0100
Stable35	Stable	-0.0408	0.0100
Stable36	Stable	-0.0408	0.0100
Stable37	Stable	-0.0408	0.0100
Stable38	Stable	-0.0408	0.0100
Stable39	Stable	-0.0408	0.0100
Stable40	Stable	-0.0408	0.0100
Stable41	Stable	-0.0408	0.0100
Stable42	Stable	-0.0408	0.0100
Stable43	Stable	-0.0408	0.0100
Stable44	Stable	-0.0408	0.0100
Stable45	Stable	-0.0408	0.0100
Stable46	Stable	-0.0408	0.0100
Stable47	Stable	-0.0408	0.0100
Stable48	Stable	-0.0408	0.0100
Stable49	Stable	-0.0408	0.0100
Stable50	Stable	-0.0408	0.0100
Stable51	Stable	-0.0408	0.0100
Stable52	Stable	-0.0407	0.0101
Stable53	Stable	-0.0408	0.0100
Stable54	Stable	-0.0408	0.0100
Stable55	Stable	-0.0408	0.0100
Stable56	Stable	-0.0408	0.0100
Stable57	Stable	-0.0408	0.0100
Stable58	Stable	-0.0408	0.0100
Stable59	Stable	-0.0408	0.0100

Stable60	Stable	-0.0408	0.0100
Stable61	Stable	-0.0408	0.0100
Stable62	Stable	-0.0408	0.0100
Stable63	Stable	-0.0408	0.0100
Stable64	Stable	-0.0408	0.0100
Stable65	Stable	-0.0408	0.0100
Stable66	Stable	-0.0408	0.0100
Stable67	Stable	-0.0408	0.0100
Stable68	Stable	-0.0408	0.0100
Stable69	Stable	-0.0408	0.0100
Stable70	Stable	-0.0408	0.0100
Stable71	Stable	-0.0408	0.0100
Stable72	Stable	-0.0408	0.0100
Stable73	Stable	-0.0408	0.0100
Stable74	Stable	-0.0408	0.0100
Stable75	Stable	-0.0408	0.0100
Stable76	Stable	-0.0404	0.0101
Stable77	Stable	-0.0408	0.0100
Stable78	Stable	-0.0408	0.0100
Stable79	Stable	-0.0408	0.0100
Stable80	Stable	-0.0408	0.0100
Stable81	Stable	-0.0408	0.0100
Stable82	Stable	-0.0399	0.0097
Stable83	Stable	-0.0408	0.0100
Stable84	Stable	-0.0408	0.0100
Stable85	Stable	-0.0408	0.0100
Stable86	Stable	-0.0408	0.0100
Stable87	Stable	-0.0408	0.0100
Stable88	Stable	-0.0408	0.0100
Stable89	Stable	-0.0408	0.0100
Stable90	Stable	-0.0408	0.0101
Stable91	Stable	-0.0404	0.0096
Stable92	Stable	-0.0408	0.0100
Stable93	Stable	-0.0408	0.0100
Stable94	Stable	-0.0408	0.0100
Stable95	Stable	-0.0408	0.0100
Stable96	Stable	-0.0408	0.0100
Stable97	Stable	-0.0408	0.0100
Stable98	Stable	-0.0388	0.0122
Stable99	Stable	-0.0408	0.0100
Stable100	Stable	-0.0408	0.0100
Stable101	Stable	-0.0408	0.0100
Stable102	Stable	-0.0408	0.0100
Stable103	Stable	-0.0408	0.0100
Stable104	Stable	-0.0408	0.0100
Stable105	Stable	-0.0408	0.0100

Stable106	Stable	-0.0408	0.0100
Stable107	Stable	-0.0408	0.0100
Stable108	Stable	-0.0408	0.0100
Stable109	Stable	-0.0408	0.0100
Stable110	Stable	-0.0402	0.0090
Stable111	Stable	-0.0408	0.0100
Stable112	Stable	-0.0408	0.0100
Stable113	Stable	-0.0403	0.0095
Stable114	Stable	-0.0408	0.0100
Stable115	Stable	-0.0408	0.0100
Stable116	Stable	-0.0408	0.0100
Stable117	Stable	-0.0408	0.0100
Stable118	Stable	-0.0408	0.0100
Stable119	Stable	-0.0408	0.0100
Stable120	Stable	-0.0408	0.0100
Stable121	Stable	-0.0408	0.0100
Stable122	Stable	-0.0408	0.0100
Stable123	Stable	-0.0408	0.0100
Stable124	Stable	-0.0408	0.0100
Stable125	Stable	-0.0408	0.0100
Stable126	Stable	-0.0408	0.0100
Stable127	Stable	-0.0408	0.0100
Stable128	Stable	-0.0408	0.0100
Stable129	Stable	-0.0408	0.0100
Stable130	Stable	-0.0408	0.0100
Stable131	Stable	-0.0408	0.0100
Stable132	Stable	-0.0408	0.0100
Stable133	Stable	-0.0408	0.0100
Stable134	Stable	-0.0408	0.0100
Stable135	Stable	-0.0408	0.0100
Stable136	Stable	-0.0408	0.0100
Stable137	Stable	-0.0408	0.0100
Stable138	Stable	-0.0408	0.0100
Stable139	Stable	-0.0408	0.0100
Stable140	Stable	-0.0405	0.0096
Stable141	Stable	-0.0408	0.0100
Stable142	Stable	-0.0408	0.0100
Stable143	Stable	-0.0408	0.0100
Stable144	Stable	-0.0408	0.0100
Stable145	Stable	-0.0408	0.0100
Stable146	Stable	-0.0408	0.0100
Stable147	Stable	-0.0408	0.0100
Stable148	Stable	-0.0408	0.0100
Stable149	Stable	-0.0408	0.0100
Stable150	Stable	-0.0408	0.0100
Stable151	Stable	-0.0408	0.0100

Stable152	Stable	-0.0408	0.0100
Stable153	Stable	-0.0408	0.0100
Stable154	Stable	-0.0408	0.0100
Stable155	Stable	-0.0408	0.0100
Stable156	Stable	-0.0373	0.0137
Stable157	Stable	-0.0408	0.0100
Stable158	Stable	-0.0408	0.0100
Stable159	Stable	-0.0408	0.0100
Stable160	Stable	-0.0408	0.0100
Stable161	Stable	-0.0408	0.0100
Stable162	Stable	-0.0408	0.0100
Stable163	Stable	-0.0408	0.0100
Stable164	Stable	-0.0408	0.0100
Stable165	Stable	-0.0408	0.0100
Stable166	Stable	-0.0408	0.0100
Stable167	Stable	-0.0408	0.0100
Stable168	Stable	-0.0408	0.0100
Stable169	Stable	-0.0408	0.0100
Stable170	Stable	-0.0408	0.0100
Stable171	Stable	-0.0408	0.0100
Stable172	Stable	-0.0408	0.0100
Stable173	Stable	-0.0399	0.0116
Stable174	Stable	-0.0408	0.0100
Stable175	Stable	-0.0408	0.0100
Stable176	Stable	-0.0408	0.0100
Stable177	Stable	-0.0408	0.0100
Stable178	Stable	-0.0408	0.0100
Stable179	Stable	-0.0408	0.0100
Stable180	Stable	-0.0408	0.0100
Stable181	Stable	-0.0408	0.0100
Stable182	Stable	-0.0408	0.0100
Stable183	Stable	-0.0408	0.0100
Stable184	Stable	-0.0408	0.0100
Stable185	Stable	-0.0408	0.0100
Stable186	Stable	-0.0408	0.0100
Stable187	Stable	-0.0408	0.0100
Stable188	Stable	-0.0408	0.0100
Stable189	Stable	-0.0408	0.0099
Stable190	Stable	-0.0408	0.0100
Stable191	Stable	-0.0408	0.0100
Stable192	Stable	-0.0408	0.0100
Stable193	Stable	-0.0408	0.0100
Stable194	Stable	-0.0408	0.0100
Stable195	Stable	-0.0408	0.0100
Stable196	Stable	-0.0408	0.0100
Stable197	Stable	-0.0408	0.0100

Stable198	Stable	-0.0408	0.0100
Stable199	Stable	-0.0408	0.0100
Stable200	Stable	-0.0408	0.0100
Aperiodic1	Aperiodic	-0.0043	-0.0327
Aperiodic2	Aperiodic	0.0571	-0.0118
Aperiodic3	Aperiodic	0.0901	-0.0052
Aperiodic4	Aperiodic	0.0298	-0.0245
Aperiodic5	Aperiodic	-0.0218	-0.0167
Aperiodic6	Aperiodic	-0.0054	-0.0240
Aperiodic7	Aperiodic	0.0617	-0.0176
Aperiodic8	Aperiodic	0.0854	0.0026
Aperiodic9	Aperiodic	0.0647	0.0002
Aperiodic10	Aperiodic	0.0342	-0.0316
Aperiodic11	Aperiodic	-0.0408	0.0099
Aperiodic12	Aperiodic	-0.0065	-0.0303
Aperiodic13	Aperiodic	0.1145	0.0434
Aperiodic14	Aperiodic	0.0439	-0.0276
Aperiodic15	Aperiodic	0.1242	0.0431
Aperiodic16	Aperiodic	0.0918	0.0211
Aperiodic17	Aperiodic	0.0705	0.0080
Aperiodic18	Aperiodic	0.1025	0.0370
Aperiodic19	Aperiodic	0.0166	-0.0355
Aperiodic20	Aperiodic	0.0124	-0.0344
Aperiodic21	Aperiodic	-0.0041	-0.0289
Aperiodic22	Aperiodic	-0.0167	-0.0240
Aperiodic23	Aperiodic	0.0744	-0.0064
Aperiodic24	Aperiodic	0.0953	0.0174
Aperiodic25	Aperiodic	0.0774	0.0011
Aperiodic26	Aperiodic	-0.0351	-0.0007
Aperiodic27	Aperiodic	-0.0185	-0.0228
Aperiodic28	Aperiodic	0.0194	-0.0254
Aperiodic29	Aperiodic	0.0144	-0.0341
Aperiodic30	Aperiodic	0.0469	-0.0118
Aperiodic31	Aperiodic	0.0879	0.0122
Aperiodic32	Aperiodic	0.0662	-0.0095
Aperiodic33	Aperiodic	0.0013	-0.0256
Aperiodic34	Aperiodic	0.0301	-0.0218
Aperiodic35	Aperiodic	0.0560	-0.0060
Aperiodic36	Aperiodic	0.0944	0.0248
Aperiodic37	Aperiodic	0.0020	-0.0339
Aperiodic38	Aperiodic	0.0996	0.0348
Aperiodic39	Aperiodic	0.0974	0.0305
Aperiodic40	Aperiodic	0.1353	0.0468
Aperiodic41	Aperiodic	0.0404	-0.0223
Aperiodic42	Aperiodic	0.0196	-0.0359
Aperiodic43	Aperiodic	0.0851	0.0115

Aperiodic44	Aperiodic	-0.0094	-0.0295
Aperiodic45	Aperiodic	0.0556	-0.0092
Aperiodic46	Aperiodic	0.0771	-0.0034
Aperiodic47	Aperiodic	0.0540	-0.0232
Aperiodic48	Aperiodic	0.0771	0.0170
Aperiodic49	Aperiodic	0.0956	0.0174
Aperiodic50	Aperiodic	0.0015	-0.0339
Aperiodic51	Aperiodic	-0.0229	-0.0185
Aperiodic52	Aperiodic	-0.0136	-0.0272
Aperiodic53	Aperiodic	0.0083	-0.0299
Aperiodic54	Aperiodic	0.0008	-0.0280
Aperiodic55	Aperiodic	0.0274	-0.0350
Aperiodic56	Aperiodic	0.0366	-0.0209
Aperiodic57	Aperiodic	0.0419	-0.0303
Aperiodic58	Aperiodic	0.0078	-0.0261
Aperiodic59	Aperiodic	0.0277	-0.0326
Aperiodic60	Aperiodic	0.0040	-0.0348
Aperiodic61	Aperiodic	0.1024	0.0306
Aperiodic62	Aperiodic	0.0330	-0.0270
Aperiodic63	Aperiodic	0.0113	-0.0262
Aperiodic64	Aperiodic	0.0030	-0.0351
Aperiodic65	Aperiodic	-0.0033	-0.0274
Aperiodic66	Aperiodic	0.0016	-0.0336
Aperiodic67	Aperiodic	0.0673	-0.0118
Aperiodic68	Aperiodic	0.0493	-0.0254
Aperiodic69	Aperiodic	0.0090	-0.0260
Aperiodic70	Aperiodic	0.0334	-0.0192
Aperiodic71	Aperiodic	0.0454	-0.0076
Aperiodic72	Aperiodic	-0.0219	-0.0192
Aperiodic73	Aperiodic	0.0194	-0.0265
Aperiodic74	Aperiodic	0.1149	0.0389
Aperiodic75	Aperiodic	-0.0160	-0.0248
Aperiodic76	Aperiodic	0.0293	-0.0336
Aperiodic77	Aperiodic	0.0189	-0.0280
Aperiodic78	Aperiodic	-0.0075	-0.0237
Aperiodic79	Aperiodic	0.0927	0.0208
Aperiodic80	Aperiodic	0.0994	0.0315
Aperiodic81	Aperiodic	0.0741	0.0003
Aperiodic82	Aperiodic	0.0561	-0.0221
Aperiodic83	Aperiodic	0.0543	-0.0225
Aperiodic84	Aperiodic	0.0027	-0.0344
Aperiodic85	Aperiodic	0.0546	-0.0181
Aperiodic86	Aperiodic	0.0969	0.0105
Aperiodic87	Aperiodic	0.0623	-0.0112
Aperiodic88	Aperiodic	0.0891	0.0194
Aperiodic89	Aperiodic	0.0696	-0.0114

Aperiodic90	Aperiodic	-0.0203	-0.0194
Aperiodic91	Aperiodic	0.0985	0.0363
Aperiodic92	Aperiodic	0.0561	-0.0223
Aperiodic93	Aperiodic	0.0870	0.0233
Aperiodic94	Aperiodic	-0.0039	-0.0269
Aperiodic95	Aperiodic	0.0063	-0.0341
Aperiodic96	Aperiodic	0.1119	0.0404
Aperiodic97	Aperiodic	0.0935	0.0225
Aperiodic98	Aperiodic	0.0768	-0.0004
Aperiodic99	Aperiodic	0.0159	-0.0358
Aperiodic100	Aperiodic	-0.0295	-0.0093
Aperiodic101	Aperiodic	-0.0284	-0.0104
Aperiodic102	Aperiodic	0.0119	-0.0352
Aperiodic103	Aperiodic	0.0837	0.0025
Aperiodic104	Aperiodic	0.1150	0.0446
Aperiodic105	Aperiodic	-0.0334	-0.0036
Aperiodic106	Aperiodic	0.1263	0.0446
Aperiodic107	Aperiodic	-0.0072	-0.0295
Aperiodic108	Aperiodic	0.0067	-0.0266
Aperiodic109	Aperiodic	0.0435	-0.0241
Aperiodic110	Aperiodic	0.0540	-0.0230
Aperiodic111	Aperiodic	-0.0136	-0.0258
Aperiodic112	Aperiodic	0.0089	-0.0349
Aperiodic113	Aperiodic	0.1144	0.0403
Aperiodic114	Aperiodic	-0.0157	-0.0215
Aperiodic115	Aperiodic	0.1091	0.0396
Aperiodic116	Aperiodic	0.0901	0.0076
Aperiodic117	Aperiodic	0.1018	0.0375
Aperiodic118	Aperiodic	0.1058	0.0346
Aperiodic119	Aperiodic	-0.0140	-0.0262
Aperiodic120	Aperiodic	0.0956	0.0085
Aperiodic121	Aperiodic	-0.0197	-0.0211
Aperiodic122	Aperiodic	0.0901	0.0103
Aperiodic123	Aperiodic	0.0929	0.0274
Aperiodic124	Aperiodic	-0.0183	-0.0231
Aperiodic125	Aperiodic	0.0857	0.0148
Aperiodic126	Aperiodic	0.0914	0.0272
Aperiodic127	Aperiodic	0.0808	0.0163
Aperiodic128	Aperiodic	0.0732	0.0100
Aperiodic129	Aperiodic	0.0290	-0.0347
Aperiodic130	Aperiodic	-0.0054	-0.0322
Aperiodic131	Aperiodic	0.0353	-0.0339
Aperiodic132	Aperiodic	0.0219	-0.0357
Aperiodic133	Aperiodic	0.0860	0.0131
Aperiodic134	Aperiodic	0.0555	-0.0220
Aperiodic135	Aperiodic	0.0219	-0.0254

Aperiodic136	Aperiodic	0.0753	0.0069
Aperiodic137	Aperiodic	0.0396	-0.0316
Aperiodic138	Aperiodic	-0.0078	-0.0308
Aperiodic139	Aperiodic	0.0411	-0.0305
Aperiodic140	Aperiodic	0.0131	-0.0299
Aperiodic141	Aperiodic	0.0410	-0.0176
Aperiodic142	Aperiodic	0.1092	0.0408
Aperiodic143	Aperiodic	0.0110	-0.0346
Aperiodic144	Aperiodic	0.0074	-0.0340
Aperiodic145	Aperiodic	0.0079	-0.0344
Aperiodic146	Aperiodic	-0.0277	-0.0122
Aperiodic147	Aperiodic	0.0386	-0.0203
Aperiodic148	Aperiodic	0.0545	-0.0151
Aperiodic149	Aperiodic	0.0379	-0.0296
Aperiodic150	Aperiodic	0.1026	0.0405
Aperiodic151	Aperiodic	-0.0084	-0.0305
Aperiodic152	Aperiodic	-0.0072	-0.0290
Aperiodic153	Aperiodic	-0.0401	0.0083
Aperiodic154	Aperiodic	0.0256	-0.0229
Aperiodic155	Aperiodic	0.0258	-0.0334
Aperiodic156	Aperiodic	0.0779	-0.0008
Aperiodic157	Aperiodic	0.0132	-0.0297
Aperiodic158	Aperiodic	0.0262	-0.0228
Aperiodic159	Aperiodic	-0.0091	-0.0303
Aperiodic160	Aperiodic	0.0052	-0.0279
Aperiodic161	Aperiodic	0.0631	-0.0172
Aperiodic162	Aperiodic	0.1101	0.0521
Aperiodic163	Aperiodic	0.0251	-0.0306
Aperiodic164	Aperiodic	0.0714	-0.0090
Aperiodic165	Aperiodic	0.0496	-0.0199
Aperiodic166	Aperiodic	0.0100	-0.0352
Aperiodic167	Aperiodic	0.0116	-0.0347
Aperiodic168	Aperiodic	0.0228	-0.0252
Aperiodic169	Aperiodic	0.0255	-0.0340
Aperiodic170	Aperiodic	0.0935	0.0309
Aperiodic171	Aperiodic	-0.0308	-0.0072
Aperiodic172	Aperiodic	0.0891	0.0276
Aperiodic173	Aperiodic	0.0676	-0.0062
Aperiodic174	Aperiodic	-0.0106	-0.0275
Aperiodic175	Aperiodic	0.0551	-0.0233
Aperiodic176	Aperiodic	0.0294	-0.0205
Aperiodic177	Aperiodic	0.0389	-0.0319
Aperiodic178	Aperiodic	0.0103	-0.0265
Aperiodic179	Aperiodic	0.0968	0.0265
Aperiodic180	Aperiodic	0.0653	-0.0027
Aperiodic181	Aperiodic	0.0282	-0.0280

Aperiodic182	Aperiodic	0.0645	-0.0146
Aperiodic183	Aperiodic	0.0697	-0.0039
Aperiodic184	Aperiodic	0.0726	-0.0092
Aperiodic185	Aperiodic	0.0484	-0.0128
Aperiodic186	Aperiodic	-0.0035	-0.0253
Aperiodic187	Aperiodic	0.0679	-0.0061
Aperiodic188	Aperiodic	-0.0218	-0.0193
Aperiodic189	Aperiodic	0.0846	0.0207
Aperiodic190	Aperiodic	-0.0188	-0.0196
Aperiodic191	Aperiodic	0.0046	-0.0340
Aperiodic192	Aperiodic	0.0685	0.0016
Aperiodic193	Aperiodic	0.0265	-0.0342
Aperiodic194	Aperiodic	-0.0220	-0.0176
Aperiodic195	Aperiodic	-0.0023	-0.0292
Aperiodic196	Aperiodic	0.0251	-0.0241
Aperiodic197	Aperiodic	0.0551	-0.0223
Aperiodic198	Aperiodic	-0.0149	-0.0253
Aperiodic199	Aperiodic	0.0906	0.0123
Aperiodic200	Aperiodic	0.1159	0.0530

Sanity checks — FPCA

Metric	Sanity-check value	Computed here
PC1	~84%	84.29%
PC2	~13%	12.67%
Cumulative	~97%	96.96%
Stable with PC1 < 0	200 / 200	200 / 200
Aperiodic with PC1 < 0	~42 / 200	42 / 200

The first two functional components explain **97.0%** of the variability among landscapes (PC1 $\approx$ 84.3%, PC2 $\approx$ 12.7%). All 200 stable series fall in the PC1 < 0 half-plane — unsurprising, since 129 of them share the exact same (identically-zero) landscape and so collapse to nearly the same point in function space. Separation is **not** perfect, though: **42 of the 200 aperiodic series** (21%) also fall in PC1 < 0, overlapping the stable cloud.

This is a genuine limitation of the fixed embedding parameters ($m = 2$, $\tau = 3$), not of the FDA pipeline: for some aperiodic trajectories, this particular embedding does not produce a topologically prominent loop — the reconstructed attractor stays close enough to a compact blob that its H1 landscape resembles a stable series' near-zero landscape. Takens' theorem guarantees a *generic* embedding recovers the attractor topology, but (m, τ) are fixed constants here, not tuned per series; a natural extension would be a sensitivity analysis over (m, τ) to check whether a higher embedding dimension or a different delay recovers the loop for these borderline series.

Functional permutation test: stable vs. aperiodic

This reuses the same vectorized max-T permutation test from Part A — already validated bit-for-bit there against `fda::tperm.fd` — evaluating each `fd` object on a 101-point grid and permuting columns of a matrix instead of re-evaluating `fd` on every permutation. The masked version below is mathematically identical to that validated function whenever every grid point's denominator is positive: that is the case for every grid point in every test Part A ever ran (its GRN landscapes never included an identically-zero one), so Part A's bit-for-bit validation carries over directly to those points. What is new here is a defensive guard for a case Part A's data never exercised.

! A defensive guard for a zero-variance denominator (does not trigger with this data)

The Welch-style test statistic at each grid point t is $|\bar{x}_1(t) - \bar{x}_2(t)|/\sqrt{v_1(t) + v_2(t)}$, and the final statistic is the supremum over t. Because 129 of the 200 stable series are identically zero, the stable group's pointwise variance $v_1(t)$ is minuscule almost everywhere, and at the very edges of the shared grid range — where essentially no series in *either* group has accumulated any persistence yet — both groups' variances could in principle hit exact floating-point zero, making the ratio the undefined form 0/0. Part A's networks never produced an identically-zero landscape, so this case never had to be guarded against there.
The guard excludes grid points with a zero denominator from the supremum, rather than let a stray `NaN` or `Inf` propagate into `max()` unnoticed:

```
den <- v1 + v2
valid <- den > 0
max(abs(m1[valid] - m2[valid]) / sqrt(den[valid]))
```

With the actual committed data, this guard turns out **not to be needed** in practice (confirmed just below, computed live from the same objects the test uses): after B-spline smoothing, the boundary points evaluate to tiny floating-point residues (of order 10^{-13} to 10^{-88}) rather than exact zero, so they pass the `den > 0` filter and contribute a small, well-defined statistic value rather than an undefined one. The guard is kept anyway, as a correctness safeguard rather than a currently-active code path: a slightly different smoothing, grid, or platform could in principle produce an exact zero, and silently propagating a `NaN` into a `max()` would be a much worse failure mode than an unused `if`.

! Stable vs. Aperiodic test result

- **Observed T_max** = 85.51 (sanity check: ~85.5)
- **Critical value (95%)** = 2.75 (sanity check: ~2.85)
- **p-value** = 0.00000 (out of 10000 permutations — a value of 0 here means p < 1e-4, not exactly zero)

With `T_max` = 85.51 — dramatically above the 95% critical value of 2.75 — the null hypothesis of functional equality between the mean stable and aperiodic H1 landscapes is rejected outright. Note that `T_max` is **not** an effect size: the supremum is reached at $t \approx 0.1844$ (mean difference $\approx$ 0.125), not at $t \approx 0.2842$ where the mean difference between regimes is actually largest (≈ 0.174, giving a statistic of only 35.95) — because the statistic is studentized, dividing by the local pooled

variance, so it is largest where the groups are both tightly concentrated *and* different, not simply where they differ most in absolute terms. The evidence for a real difference is the p-value and the mean-landscape comparison (Figure 2 above), not the raw scale of `T_max`. As noted above, the zero-variance guard was not actually needed here: 101 of 101 grid points are valid for this test (computed live from the same objects `vtperm` uses), and the within-regime control splits below are likewise always fully valid.

This is expected given how the two regimes were constructed (one produces topological loops, the other does not), but the result itself is new: `Example1Beetles.Rmd` never asked this question in a functional-hypothesis-testing sense. It runs two independent classification checks against the true regime labels — k-means clustering on the raw series (`cfm`) and a rule based on which homology dimension each series' single most persistent bar belongs to (`cfm_hom`) — and reports each as its own confusion matrix; the two checks are never compared against each other, and neither is a test of functional equality between group means. The permutation test here instead directly tests whether the *entire mean landscape curves* differ, which is the same kind of claim Part A makes about asymptomatic vs. symptomatic GRNs.

Control analysis (within-regime calibration)

Part A's split-half validation enumerated *every* possible partition of each group (`choose(9,4) = 126`, `choose(8,4) = 70`) because those groups were small enough to do so exhaustively. Here each regime has 200 series, and `choose(200, 100)` is on the order of 10^{58} — completely infeasible to enumerate. Instead, this is a **random sample of 200 balanced (100 vs. 100) splits per regime**, drawn with a fixed seed continuing directly from the main test's RNG stream above (not "all partitions", unlike Part A). Because these splits are all drawn from the same 200 series per regime (not 200 independent fresh samples), they are correlated with each other, so the mean/sd/fraction below understate the true sampling uncertainty somewhat — the same caveat applies, in a milder form, to Part A's exhaustive enumeration, since overlapping subsets there are not independent either.

> **i** The stable control is not as informative as the aperiodic one
>
> The two regimes' control analyses are not equivalent evidence. 129 of the 200 stable series (64%) are identically zero, so a random 100-vs-100 split of the stable regime compares two halves that are, on average, **64%** identical zero curves each (computed live across the 200 splits actually drawn above) — it is substantially a "zeros vs. zeros" comparison, well calibrated close to trivially rather than as a demonstration that the test correctly handles genuine functional variability. The zero-variance guard from the previous section is, in fact, never triggered in these splits either: all 101 of 101 grid points are always valid (computed live, same as the main test above). The **aperiodic control is the informative one**: no aperiodic series is identically zero, so its 200 splits compare genuinely varying curves on the full, unmasked grid (101 of 101 points valid there too, for the unrelated reason that no aperiodic series has zero variance in the first place) — its calibration result is the one that should be read as evidence the test does not over-reject.

Table 2: Summary of the within-regime control analysis

Regime	mean p-value	sd(p-value)	% splits with $p < 0.05$
Stable (200 random splits, 100 vs 100)	0.534	0.297	5.0%
Aperiodic (200 random splits, 100 vs 100)	0.505	0.282	4.0%

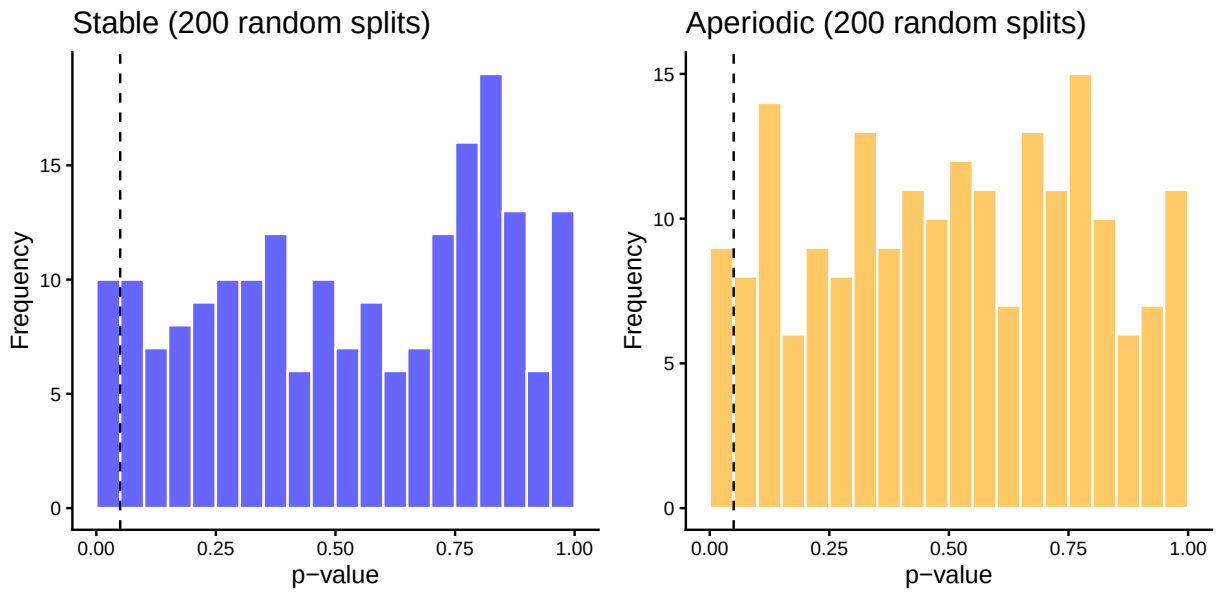

Figure 4: Distribution of control-analysis p-values, by regime (dashed line at $p = 0.05$).

Sanity checks — control analysis

Regime	mean p	sd p	% $p < 0.05$
Stable	0.534	0.297	5.0%
Aperiodic	0.505	0.282	4.0%

Both regimes' p-values are consistent with the nominal 5% false-positive rate expected under the null, but as noted above, only the **aperiodic** result is strong evidence that the test does not over-reject on genuinely varying functional data — the stable result is largely a check that comparing zero to zero correctly fails to reject, which is a much lower bar. Taken together, they still calibrate the significance of the stable-vs-aperiodic result above: the rejection there reflects a real difference between regimes, not a general tendency of the test to over-reject.

The approach-to-equilibrium transient

Everything above uses `beetles_landscapes.ipynb` at its default `BURN_IN = 0`: the retained 201-step series starts right after the random initial condition, not after the population has settled onto its attractor — matching `Example1Beetles.Rmd`, which this chapter replicates and which never discards an initial transient either. This section makes that choice explicit. It (1) derives, in closed form, *why* the stable regime's approach to equilibrium registers as an H1 loop at all; (2) corrects a claim that is easy to make too quickly from the structural check above — an identically-zero *landscape* is not the same thing as an empty H1 *diagram*; and (3) quantifies, via a dedicated `BURN_IN = 200` sensitivity run, how much of the topological signal is attributable to the transient, and why discarding it would have left the main analysis without a working calibration check. None of this changes the result above; it explains it.

The mechanism: a damped spiral onto the fixed point

The stable regime (`u_a = 0.73`) is a point attractor: the deterministic LPA map has a single fixed point that every trajectory in this regime converges to. Finding it by direct iteration and linearising there (the Jacobian of the LPA map) characterises *how* that convergence happens:

```
B_    <- 7.48; C_EA_ <- 0.009; C_PA_ <- 0.004; C_EL_ <- 0.012; U_L_ <- 0.267
U_A_STABLE_ <- 0.73

lpa_step_ <- function(state, u_a) {
  L <- state[1]; P <- state[2]; A <- state[3]
  c(
    B_ * A * exp(-C_EL_ * L - C_EA_ * A),
    L * (1 - U_L_),
    P * exp(-C_PA_ * A) + A * (1 - u_a)
  )
}

# The map is a contraction at the fixed point (see the Jacobian below), so
# direct iteration from an arbitrary interior starting point converges to
# machine precision regardless of where it starts.
state <- c(50, 30, 60)
for (i in 1:5000) state <- lpa_step_(state, U_A_STABLE_)
fixed_point <- state
names(fixed_point) <- c("L", "P", "A")
fixed_point
```

```
       L        P        A
92.21230 67.59162 69.98357
```

```
lpa_jacobian_ <- function(state, u_a) {
  L <- state[1]; P <- state[2]; A <- state[3]
  e <- exp(-C_EL_ * L - C_EA_ * A)
```

```
  matrix(c(
    -C_EL_ * B_ * A * e, 0,                  B_ * e - C_EA_ * B_ * A * e,
    1 - U_L_,            0,                  0,
    0,                   exp(-C_PA_ * A),    -C_PA_ * P * exp(-C_PA_ * A) + (1 - u_a)
  ), nrow = 3, byrow = TRUE)
}

eig <- eigen(lpa_jacobian_(fixed_point, U_A_STABLE_))$values
eig
```

```
[1] -0.7457858+0.2082557i -0.7457858-0.2082557i  0.4506720+0.0000000i
```

```
Mod(eig)
```

```
[1] 0.774317 0.774317 0.450672
```

```
lambda_complex   <- eig[abs(Im(eig)) > 1e-8][1]
rotation_period  <- 2 * pi / abs(Arg(lambda_complex))
contraction_step <- Mod(lambda_complex)
```

The fixed point is `(L, P, A)` $\approx$ (92.212, 67.592, 69.984), and the Jacobian there has a **complex-conjugate eigenvalue pair** ($\lambda \approx$ -0.7458 $\pm$ 0.2083i, modulus $\approx$ 0.7743) alongside a real eigenvalue ($\approx$ 0.4507). A complex eigenvalue pair means the linearised dynamics near the fixed point are a **rotation composed with a contraction**: every trajectory spirals in rather than approaching monotonically. The rotation completes a full turn every $2\pi/|\mathrm{Arg}(\lambda)| \approx 2.19$ steps, while each step shrinks the distance to the fixed point by a factor of $|\lambda| \approx 0.7743$.

This matters for persistent homology specifically because **a spiral is a closed curve** when it is embedded in a delay-coordinate phase space: over one rotation period the trajectory loops most of the way around the fixed point before contracting to the next, smaller loop. Takens embedding turns this into a point cloud that traces something topologically indistinguishable, at the resolution of a single loop, from a genuine cycle — so Vietoris–Rips persistent homology registers an H1 feature for it, even though the spiral is not part of the attractor itself and vanishes (in principle, at infinite resolution) as $t \to \infty$. The stable regime's attractor, the fixed point alone, truly has no loops; the finite-length *approach* to it does.

The corrected claim: absence on the grid, not absence of homology

The structural check above found 129 of 200 stable landscapes identically zero on the shared evaluation grid. Read on its own, that number invites the conclusion "these series have no H1 features" — but that conclusion is wrong. Every stable series still carries the decaying spiral just derived, which is a genuine (if short-lived) H1 loop; the other 71 of 200 (36%) already show it directly, with a landscape that survives on the grid. A prior diagnostic run of `regime_diagnostics.py` (committed as `regime_diagnostics_output.txt`) makes the resolution effect explicit: over a fresh no-burn-in

simulation, **every** stable series' H1 diagram was non-empty, yet the median of each series' maximum H1 lifetime was only `0.00022` — well below the shared grid's point spacing, which is 0.00154 for the actual committed grid used throughout this document (`0.00153` for that diagnostic run's own grid). A persistence landscape evaluates to zero at grid point t whenever no birth–death interval contains t; a feature with lifetime shorter than the grid spacing can fail to contain *any* grid point purely by bad luck of where the birth falls relative to the nearest sample, independent of whether the feature is topologically real. So: the feature exists, and is invisible to this particular 500-point sampling, not absent. This is a sampling-resolution statement, not a homological one.

Sensitivity run: discarding the transient (`BURN_IN = 200`)

To isolate the two **attractors** — comparing "fixed point" against "closed orbit" instead of "fixed point + approach spiral" against "closed orbit + approach transient" — `beetles_landscapes.ipynb` can be re-run with `BURN_IN = 200` (an *additional* 200 steps simulated and discarded before the retained 201-step series begins, so series length, embedding and grid are unchanged). That run is committed in `output_burnin/`, analysed here with exactly the same pipeline used for the main analysis above.

Metric	Main (no burn-in)	Burn-in appendix
Stable landscapes identically zero	129 / 200	200 / 200
Mean aperiodic landscape peak	0.2274	0.3492
PC1 / PC2 variance explained	84.3% / 12.7%	99.0% / 0.7%
Aperiodic with PC1 < 0	42 / 200	9 / 200
Stable vs. aperiodic T_{max} (crit95)	85.51 (2.75)	198.65 (2.58)
Within-stable control	defined (mean p $\approx$ 0.53)	undefined (degenerate)

Discarding the transient does exactly what removing an approach spiral should do: every stable landscape collapses to identically zero (200/200, versus 129/200 with the transient retained), PC1 alone explains essentially all the variance (99.0%), and the stable-vs-aperiodic separation gets *sharper*, not weaker (T_{max} 198.65 versus 85.51; aperiodic overlap into PC1 < 0 drops to 9/200 from 42/200).

The compensation. That sharper separation comes at a real cost: with every stable landscape collapsed to the same identically-zero curve, a within-stable split has **zero pooled variance at every grid point**, so `vtperm()`'s `NA_real_` guard (kept and explained in the code above) triggers for all 200 splits, and the within-stable calibration control that works in the main analysis becomes mathematically undefined here — there is no variability left to split. This is precisely why the main analysis above retains the transient: it is the only version of this pipeline in which the within-stable control is a meaningful check rather than an ungrounded "compare zero to zero" tautology, and losing that check to gain a cleaner-looking separation was judged not worth it, especially since the separation was already decisive (`p < 1e-4`) with the transient retained.

Where the transient's effect kicks in: a burn-in sweep

The two runs above are single points (`BURN_IN` 0 and 200) on a continuum. `beetles_landscapes.ipynb` sweeps a grid of burn-in values, each with a smaller 60-series-per-regime cohort (fixed seed, same model), recording how many stable series have a genuinely empty H1 diagram and the aperiodic regime's mean landscape peak, written to `output/burnin_sweep.csv`:

```
sweep <- read.csv("output/burnin_sweep.csv")
sweep$pct_stable_h1_empty <- sprintf("%.0f%%", 100 * sweep$n_stable_h1_empty / sweep$n_sweep)
knitr::kable(
  data.frame(
    `Burn-in`                  = sweep$burn_in,
    `Stable, H1 empty`         = sprintf("%d / %d (%s)", sweep$n_stable_h1_empty, sweep$n_swe
    `Mean aperiodic peak`      = sprintf("%.4f", sweep$mean_aperiodic_peak),
    check.names = FALSE
  ),
  align = "lrr"
)
```

Burn-in	Stable, H1 empty	Mean aperiodic peak
0	0 / 60 (0%)	0.2304
10	0 / 60 (0%)	0.2655
25	0 / 60 (0%)	0.2844
50	0 / 60 (0%)	0.2986
100	0 / 60 (0%)	0.3161
120	11 / 60 (18%)	0.3213
140	59 / 60 (98%)	0.3264
150	60 / 60 (100%)	0.3296
160	60 / 60 (100%)	0.3328
200	60 / 60 (100%)	0.3435
400	60 / 60 (100%)	0.3666
800	60 / 60 (100%)	0.3715

The empirical transition is abrupt, not gradual: essentially none of the 60 stable series have an empty H1 diagram through burn-in 100, then almost all of them do by burn-in 140–150. That matches the mechanism derived above: because the stable regime's approach is a *contraction*, its transient decays geometrically at rate $|\lambda| \approx 0.7743$ per step, so there is a fairly sharp burn-in past which the spiral's remaining radius falls below what any trajectory in this system can resolve — not a slow fade. Equating "remaining radius has shrunk by 16 orders of magnitude" ($16 \ln 10$, an arbitrary but generous margin) with n steps of that contraction gives a threshold of $16\ln(10)/(-\ln|\lambda|) \approx 144$ steps — close to where the sweep actually crosses over.

The aperiodic regime's mean landscape peak, by contrast, is still visibly increasing at burn-in 200 (0.3435) and has not fully leveled off even by burn-in 800 (0.3715). This is consistent with the aperiodic regime's own approach dynamics: `regime_diagnostics.py`'s Lyapunov spectrum for `u_a = 0.96` gives a near-zero exponent along the invariant curve itself (motion *on* the attractor does

not decay) and a much smaller contracting exponent transverse to it, ≈ -0.0167, versus the stable regime's $-\ln|\lambda| \approx 0.2558$ derived above — the aperiodic transient decays roughly 15.3 times more slowly per step than the stable one, not the "about 40 times" this document's brief anticipated; that number is reported here as computed, flagged as a deviation from that expectation rather than adjusted to match it. At either figure, `BURN_IN = 200` sits well short of full convergence for the aperiodic regime, even though it is already far past the stable regime's threshold.

Conclusion

Discarding the transient sharpens an already-decisive result; it does not create it. With the transient retained (the main analysis above), the stable-vs-aperiodic separation is significant at `p < 1e-4` with 42 of 200 aperiodic series still overlapping the stable cloud on PC1 — a real but incomplete separation, honestly reported. The burn-in appendix quantifies the alternative: removing the transient pushes that overlap down to 9/200 and sharpens T_{max}, at the cost of making the within-stable calibration control undefined. Between a replica that keeps the heritage pipeline's transient and a variant that discards it for a cleaner-looking (but uncalibrated) result, the main analysis keeps the transient — consistent with the brief received from the thesis director, who asked that dropping it be discussed, not adopted outright.

Appendix

Session information

```
sessionInfo()
```

```
R version 4.5.3 (2026-03-11)
Platform: aarch64-apple-darwin20
Running under: macOS Tahoe 26.5.2

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LA

locale:
[1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8

time zone: Europe/Madrid
tzcode source: internal

attached base packages:
[1] splines   stats     graphics  grDevices utils     datasets  methods
[8] base

other attached packages:
[1] patchwork_1.3.2 ggplot2_4.0.3   fda_6.3.0       deSolve_1.42
```

```
[5] fds_1.9         RCurl_1.98-1.19 rainbow_3.8     pcaPP_2.0-5
[9] MASS_7.3-65

loaded via a namespace (and not attached):
 [1] Matrix_1.7-4       gtable_0.3.6       jsonlite_2.0.0     dplyr_1.2.1
 [5] compiler_4.5.3     tidyselect_1.2.1   ks_1.15.2          bitops_1.0-9
 [9] cluster_2.1.8.2    scales_1.4.0       yaml_2.3.12        fastmap_1.2.0
[13] lattice_0.22-9     R6_2.6.1           labeling_0.4.3     generics_0.1.4
[17] knitr_1.51         tibble_3.3.1       pillar_1.11.1      RColorBrewer_1.1-3
[21] rlang_1.2.0        hdrcde_3.5.0       xfun_0.57          S7_0.2.2
[25] cli_3.6.6          withr_3.0.2        magrittr_2.0.5     digest_0.6.39
[29] grid_4.5.3         mvtnorm_1.4-2      mclust_6.1.3       lifecycle_1.0.5
[33] vctrs_0.7.3        KernSmooth_2.23-26 evaluate_1.0.5     pracma_2.4.6
[37] glue_1.8.1         farver_2.1.2       colorspace_2.1-3   rmarkdown_2.31
[41] pkgconfig_2.0.3    tools_4.5.3        htmltools_0.5.9
```

Full code (reproducible)

```r
library(fda)
library(ggplot2)
library(patchwork)

# Reference values from a prior direct computation on the committed landscape
# matrices (for sanity-check comparison only; every number reported in the
# body is recomputed here from output/*.csv, not hardcoded from this list).
# output/ holds the ORIGINAL run (beetles_landscapes.ipynb at its default
# BURN_IN = 0), faithful to the heritage code in Example1Beetles.Rmd, which
# never discards the initial transient -- this is the MAIN analysis, both
# because this chapter is a replica of that code and because the thesis
# director asked to DISCUSS dropping the transient, not to adopt it as the
# primary pipeline. output_burnin/ holds the BURN_IN = 200 run, used only as
# a supporting sensitivity analysis in "The approach-to-equilibrium
# transient" below. The p-value and critical value carry Monte Carlo noise
# and may differ in the last reported digit after re-rendering.
ref <- list(
  n_stable_zero = 129, n_aperiodic_zero = 0,
  mean_max_stable = 0.0016, mean_max_aperiodic = 0.227,
  pc1_pct = 84, pc2_pct = 13,
  n_aperiodic_pc1_neg = 42,
  test1_Tobs = 85.5, test1_crit95 = 2.85,
  # With the transient retained, the stable regime is NOT uniformly zero (71
  # of 200 landscapes carry genuine H1 signal above the grid's resolution),
  # so a within-stable split has positive pooled variance almost everywhere
  # and the control statistic is well defined.
  stable_control_degenerate = FALSE
)
```

```r
# Reference values for the burn-in = 200 run in output_burnin/, reported only
# in "The approach-to-equilibrium transient" section below as a supporting
# sensitivity analysis, not the main result.
ref_burnin <- list(
  n_stable_zero = 200, mean_max_aperiodic = 0.349,
  pc1_pct = 99, pc2_pct = 1,
  n_aperiodic_pc1_neg = 9,
  test1_Tobs = 198.65, test1_crit95 = 2.58
)

col_stable    <- "blue"
col_aperiodic <- "orange"

# The memoria lives outside this repository, so a standalone clone must
# still render cleanly: figures are exported to it only when the target
# directory is actually present (checked at each ggsave() call site).
fig_dir <- "../../TFM_UCD/figures"
tseq      <- as.matrix(read.csv("output/tseq.csv", header = FALSE))[, 1]
DataStable    <- as.matrix(read.csv("output/stable_landscapes.csv", header = FALSE))
DataAperiodic <- as.matrix(read.csv("output/aperiodic_landscapes.csv", header = FALSE))
DataBoth  <- rbind(DataStable, DataAperiodic)

stopifnot(
  "Expected 200 stable series"    = nrow(DataStable) == 200,
  "Expected 200 aperiodic series" = nrow(DataAperiodic) == 200,
  "Expected a 500-point shared grid" = length(tseq) == 500,
  "Landscape matrices must have one column per grid point" =
    ncol(DataStable) == length(tseq) && ncol(DataAperiodic) == length(tseq)
)
is_zero_row <- function(M) apply(M, 1, function(r) all(r == 0))
n_stable_zero    <- sum(is_zero_row(DataStable))
n_aperiodic_zero <- sum(is_zero_row(DataAperiodic))

stopifnot(
  "Expected 129 of 200 stable landscapes identically zero on the grid (a grid-resolution effect
    n_stable_zero == ref$n_stable_zero,
  "Expected 0 aperiodic series with an identically-zero landscape" =
    n_aperiodic_zero == 0
)

mean_max_stable    <- mean(apply(DataStable, 1, max))
mean_max_aperiodic <- mean(apply(DataAperiodic, 1, max))
tol_meanmax <- 0.02
stopifnot(
  "Mean stable landscape peak out of tolerance"    = abs(mean_max_stable - ref$mean_max_stable)
  "Mean aperiodic landscape peak out of tolerance" = abs(mean_max_aperiodic - ref$mean_max_aper
)
```

```r
create_fd <- function(Data, tseq) {
  basis <- create.bspline.basis(range(tseq), nbasis = ncol(Data) - 1, norder = 2)
  fdobj <- smooth.basis(tseq, t(Data), basis)
  fdobj$fd
}

fdStable    <- create_fd(DataStable, tseq)
fdAperiodic <- create_fd(DataAperiodic, tseq)
fdBoth      <- create_fd(DataBoth, tseq)

pca <- pca.fd(fdBoth, nharm = 2)
pc1_pct <- pca$varprop[1] * 100
pc2_pct <- pca$varprop[2] * 100
cum_pct <- pc1_pct + pc2_pct

# An eigenvector's sign is mathematically arbitrary (pca.fd can return either
# sign depending on the LAPACK/fda version), so PC1 alone is not a stable
# quantity to assert on across environments. Fix it deterministically here,
# before computing any score or assert, to the convention "stable series
# score negative on PC1" - an independent PCA run on these same arrays has
# been observed to return the opposite sign, so this is not hypothetical.
pc1_raw   <- pca$scores[, 1]
sign_fix  <- if (mean(pc1_raw[1:200]) > 0) -1 else 1  # 1:200 = stable rows
PC1 <- sign_fix * pc1_raw
PC2 <- pca$scores[, 2]  # PC2's sign is independently arbitrary but no assert depends on it

ids   <- c(paste0("Stable", 1:200), paste0("Aperiodic", 1:200))
group <- c(rep("Stable", 200), rep("Aperiodic", 200))

pca_df <- data.frame(
  PC1 = PC1,
  PC2 = PC2,
  ID = ids,
  Group = group
)

n_stable_pc1_neg    <- sum(pca_df$PC1[pca_df$Group == "Stable"] < 0)
n_aperiodic_pc1_neg <- sum(pca_df$PC1[pca_df$Group == "Aperiodic"] < 0)
knitr::kable(
  data.frame(
    Component = c("PC1", "PC2", "Cumulative (PC1+PC2)"),
    `% variance explained` = c(sprintf("%.2f%%", pc1_pct), sprintf("%.2f%%", pc2_pct), sprintf
    check.names = FALSE
  ),
  align = "lr"
)
fpca_score_plot <- ggplot(pca_df, aes(x = PC1, y = PC2, color = Group)) +
```

```r
  geom_hline(yintercept = 0, linetype = "dashed", color = "black") +
  geom_vline(xintercept = 0, linetype = "dashed", color = "black") +
  geom_point(size = 2, alpha = 0.6) +
  scale_color_manual(values = c("Stable" = col_stable, "Aperiodic" = col_aperiodic)) +
  labs(x = "PC1 Score", y = "PC2 Score", title = "") +
  theme_classic(base_size = 16) +
  theme(
    legend.position = "right",
    legend.title = element_blank(),
    plot.title = element_text(hjust = 0.5, face = "bold")
  )
fpca_score_plot
# Exported for the memoria (TFM_UCD/figures/ts_fpca_scores.pdf): with the
# burn-in, all 200 stable series share the same identically-zero landscape,
# so their scores collapse onto a single point in the PC1 < 0 half-plane.
if (dir.exists(fig_dir)) {
  ggsave(file.path(fig_dir, "ts_fpca_scores.pdf"), fpca_score_plot,
         width = 7, height = 5)
}
pca_full_df <- data.frame(
  ID = pca_df$ID,
  Group = pca_df$Group,
  PC1 = round(pca_df$PC1, 4),
  PC2 = round(pca_df$PC2, 4)
)
knitr::kable(pca_full_df, align = "llrr")
tol_fpca <- 2  # percentage points - this is a new analysis, not a reproduction, hence a loose
stopifnot(
  "PC1 far from sanity-check value"    = abs(pc1_pct - ref$pc1_pct) < tol_fpca,
  "PC2 far from sanity-check value"    = abs(pc2_pct - ref$pc2_pct) < tol_fpca,
  "Expected all 200 stable series to have PC1 < 0" = n_stable_pc1_neg == 200,
  "Aperiodic PC1<0 count far from sanity-check value" = abs(n_aperiodic_pc1_neg - ref$n_aperio
)
rowVar <- function(M) {
  n <- ncol(M)
  mu <- rowMeans(M)
  rowSums((M - mu)^2) / (n - 1)
}

vtperm <- function(X, i1, i2, nperm = 10000) {
  x1 <- X[, i1, drop = FALSE]
  x2 <- X[, i2, drop = FALSE]
  n1 <- ncol(x1); n2 <- ncol(x2)
  Xmat <- cbind(x1, x2)
  n <- n1 + n2
  Ts <- function(M) {
    m1 <- rowMeans(M[, 1:n1, drop = FALSE])
```

```
    m2 <- rowMeans(M[, n1 + (1:n2), drop = FALSE])
    v1 <- rowVar(M[, 1:n1, drop = FALSE]) / n1
    v2 <- rowVar(M[, n1 + (1:n2), drop = FALSE]) / n2
    den <- v1 + v2
    valid <- den > 0
    # Fully degenerate case: if no grid point has positive pooled variance the
    # statistic is undefined, not -Inf. This does NOT trigger in the main
    # analysis below: with the transient retained, 71 of the 200 stable
    # landscapes carry genuine (if grid-sampled-as-zero) H1 signal, and a
    # within-stable split has positive pooled variance almost everywhere. It
    # is a correctness safeguard that DOES activate in "The
    # approach-to-equilibrium transient" section's burn-in appendix, where
    # discarding the transient makes every stable landscape identically zero
    # and a within-stable split compares constant curves against constant
    # curves. Returning NA_real_ keeps that visible instead of letting max()
    # of an empty vector propagate -Inf silently.
    if (!any(valid)) return(NA_real_)
    max(abs(m1[valid] - m2[valid]) / sqrt(den[valid]))
  }
  Tobs <- Ts(Xmat)
  if (is.na(Tobs)) {
    return(list(pval = NA_real_, Tobs = NA_real_, crit95 = NA_real_,
                degenerate = TRUE))
  }
  Tn <- numeric(nperm)
  for (i in 1:nperm) Tn[i] <- Ts(Xmat[, sample(n)])
  list(pval = mean(Tobs < Tn), Tobs = Tobs, crit95 = unname(quantile(Tn, 0.95)),
       degenerate = FALSE)
}
count_valid <- function(v1, v2) sum((v1 + v2) > 0)

# Runs the main stable-vs-aperiodic test plus both within-regime control
# analyses, with an on-disk cache keyed on a hash of its own input CSVs (if
# the committed CSVs ever change, the stored p-values would otherwise
# silently go stale and asserts would pass against outdated numbers instead
# of catching the change; tools::md5sum is base R, no extra dependency).
# Factored out so the SAME code path runs both the main analysis (output/,
# below) and the burn-in appendix (output_burnin/, in "The
# approach-to-equilibrium transient" section) -- the two differ only in
# which CSVs they read, not in how the statistic is computed.
run_permtest_suite <- function(DataStable, fdStable, fdAperiodic, tseq,
                               input_files, cache_file, n_splits = 200) {
  input_hash  <- paste(unname(tools::md5sum(input_files)), collapse = "_")
  cached      <- if (file.exists(cache_file)) readRDS(cache_file) else NULL
  cache_valid <- !is.null(cached) && identical(cached$input_hash, input_hash)

  if (!cache_valid) {
```

```r
set.seed(123)
argvals <- seq(min(tseq), max(tseq), length.out = 101)

# 1) Main test: Stable vs Aperiodic
XFull <- cbind(eval.fd(argvals, fdStable), eval.fd(argvals, fdAperiodic))
test1 <- vtperm(XFull, 1:200, 201:400, nperm = 10000)

# Diagnostics for the main test (descriptive only; does not affect test1
# or consume any random numbers, so it cannot change which permutations
# get drawn below).
m1_diag <- rowMeans(XFull[, 1:200]); m2_diag <- rowMeans(XFull[, 201:400])
v1_diag <- rowVar(XFull[, 1:200]) / 200; v2_diag <- rowVar(XFull[, 201:400]) / 200
den_diag <- v1_diag + v2_diag
diff_diag <- abs(m1_diag - m2_diag)
stat_diag <- ifelse(den_diag > 0, diff_diag / sqrt(den_diag), NA_real_)
i_sup     <- which.max(stat_diag)
i_diffmax <- which.max(diff_diag)
test1_diag <- list(
  n_valid = count_valid(v1_diag, v2_diag), n_grid = length(den_diag),
  t_sup = argvals[i_sup], diff_sup = diff_diag[i_sup], stat_sup = stat_diag[i_sup],
  t_diffmax = argvals[i_diffmax], diff_diffmax = diff_diag[i_diffmax], stat_diffmax = stat
)

# 2) Control analysis, stable regime: exhaustive enumeration of all 4-vs-4-
#    style splits (as in Part A) is infeasible here (choose(200,100) is
#    astronomically large), so instead draw a random sample of 200 balanced
#    (100 vs 100) splits. Alongside each split's p-value, also record (a)
#    how many of the 101 grid points survive the zero-variance mask and
#    (b) what fraction of each half's 100 series is identically zero -
#    neither consumes random numbers, so neither changes pvalsStable.
XStable  <- eval.fd(argvals, fdStable)
pvalsStable     <- numeric(n_splits)
validStable     <- integer(n_splits)
fracZeroStable  <- numeric(n_splits)
for (i in seq_len(n_splits)) {
  g1 <- sample(1:200, 100); g2 <- setdiff(1:200, g1)
  pvalsStable[i] <- vtperm(XStable, g1, g2, nperm = 10000)$pval
  vv1 <- rowVar(XStable[, g1]) / 100; vv2 <- rowVar(XStable[, g2]) / 100
  validStable[i] <- count_valid(vv1, vv2)
  fracZeroStable[i] <- mean(c(sum(is_zero_row(DataStable[g1, , drop = FALSE])),
                              sum(is_zero_row(DataStable[g2, , drop = FALSE])))) / 100
}

# 3) Control analysis, aperiodic regime: same random-sample approach and
#    the same grid-validity diagnostic (no aperiodic series is identically
#    zero, so there is no zero-fraction to report here).
XAperiodic <- eval.fd(argvals, fdAperiodic)
```

```
    pvalsAperiodic <- numeric(n_splits)
    validAperiodic <- integer(n_splits)
    for (i in seq_len(n_splits)) {
      g1 <- sample(1:200, 100); g2 <- setdiff(1:200, g1)
      pvalsAperiodic[i] <- vtperm(XAperiodic, g1, g2, nperm = 10000)$pval
      vv1 <- rowVar(XAperiodic[, g1]) / 100; vv2 <- rowVar(XAperiodic[, g2]) / 100
      validAperiodic[i] <- count_valid(vv1, vv2)
    }

    cached <- list(
      input_hash = input_hash, test1 = test1, test1_diag = test1_diag,
      pvalsStable = pvalsStable, pvalsAperiodic = pvalsAperiodic,
      validStable = validStable, validAperiodic = validAperiodic,
      fracZeroStable = fracZeroStable
    )
    saveRDS(cached, cache_file)
  }

  cached
}
res_main <- run_permtest_suite(
  DataStable, fdStable, fdAperiodic, tseq,
  input_files = c("output/tseq.csv", "output/stable_landscapes.csv", "output/aperiodic_landsca
  cache_file  = "output/results_partB_permtest_cache.rds"
)

test1          <- res_main$test1
test1_diag     <- res_main$test1_diag
pvalsStable    <- res_main$pvalsStable
pvalsAperiodic <- res_main$pvalsAperiodic
validStable    <- res_main$validStable
validAperiodic <- res_main$validAperiodic
fracZeroStable <- res_main$fracZeroStable

stopifnot(
  "Expected 200 stable control splits"    = length(pvalsStable) == 200,
  "Expected 200 aperiodic control splits" = length(pvalsAperiodic) == 200
)

# With the transient retained, most stable landscapes carry genuine (if
# grid-sampled-as-zero) H1 signal, so a within-stable split should NOT be
# degenerate here -- this flag exists so the same vtperm()/fmt() machinery
# below can be reused unchanged for the burn-in appendix in "The
# approach-to-equilibrium transient" section, where discarding the transient
# DOES make every within-stable split degenerate (every stable landscape
# becomes identically zero there, not just a grid-resolution artefact).
stable_control_degenerate <- all(is.na(pvalsStable))
```

```r
# Formatting helper: report "undefined" where the statistic does not exist
# (this triggers for the stable control in the burn-in appendix below, not
# in the main analysis here).
fmt <- function(x, degenerate, pct = FALSE) {
  if (isTRUE(degenerate)) return("undefined")
  sprintf(if (pct) "%.1f%%" else "%.3f", x)
}
stopifnot(
  "Stable control degeneracy does not match the reference expectation" =
    identical(stable_control_degenerate, ref$stable_control_degenerate),
  "Aperiodic control must remain well defined" = !any(is.na(pvalsAperiodic))
)
tol_Tobs   <- 15   # loose: Tobs is a studentized statistic, sensitive to which grid point the
tol_crit95 <- 0.5  # tighter: this is a quantile of 10000 permutation draws, not a studentized
stopifnot(
  "Observed Tmax far from sanity-check value"  = abs(test1$Tobs - ref$test1_Tobs) < tol_Tobs,
  "95% critical value far from sanity-check value" = abs(test1$crit95 - ref$test1_crit95) < to
  "Expected a highly significant result (p < 0.01)" = test1$pval < 0.01
)
splithalf_summary <- data.frame(
  Regime = c("Stable (200 random splits, 100 vs 100)", "Aperiodic (200 random splits, 100 vs 1
  # fmt() prints "undefined" rather than NA for the degenerate stable control,
  # so the table states why there is no number instead of showing a blank.
  `mean p-value` = c(fmt(mean(pvalsStable, na.rm = TRUE), stable_control_degenerate),
                     sprintf("%.3f", mean(pvalsAperiodic))),
  `sd(p-value)` = c(fmt(sd(pvalsStable, na.rm = TRUE), stable_control_degenerate),
                    sprintf("%.3f", sd(pvalsAperiodic))),
  `% splits with p < 0.05` = c(fmt(mean(pvalsStable < 0.05, na.rm = TRUE) * 100,
                                   stable_control_degenerate, pct = TRUE),
                               sprintf("%.1f%%", mean(pvalsAperiodic < 0.05) * 100)),
  check.names = FALSE
)
knitr::kable(splithalf_summary, align = "lrrr")
histStable <- ggplot(data.frame(p = pvalsStable), aes(x = p)) +
  geom_histogram(binwidth = 0.05, boundary = 0, fill = col_stable, alpha = 0.6, color = "white
  geom_vline(xintercept = 0.05, linetype = "dashed", color = "black") +
  labs(title = "Stable (200 random splits)", x = "p-value", y = "Frequency") +
  theme_classic(base_size = 13)

histAperiodic <- ggplot(data.frame(p = pvalsAperiodic), aes(x = p)) +
  geom_histogram(binwidth = 0.05, boundary = 0, fill = col_aperiodic, alpha = 0.6, color = "wh
  geom_vline(xintercept = 0.05, linetype = "dashed", color = "black") +
  labs(title = "Aperiodic (200 random splits)", x = "p-value", y = "Frequency") +
  theme_classic(base_size = 13)

histStable + histAperiodic
```

```r
# Exported for the memoria (TFM_UCD/figures/ts_splithalf_hist.pdf): only the
# aperiodic panel, since the stable control is degenerate (every split
# compares identically-zero curves, so its p-value is undefined, not a
# distribution) and has nothing to plot.
if (dir.exists(fig_dir)) {
  ggsave(file.path(fig_dir, "ts_splithalf_hist.pdf"), histAperiodic,
         width = 7, height = 4.5)
}
tol_mean <- 0.05
tol_frac <- 5  # percentage points - 200 random splits is a noisier estimate than Part A's exha
stopifnot(
  # The stable control is skipped when degenerate: with no signal to detect
  # there is no calibration to check, and the aperiodic control carries the
  # burden of demonstrating that the test is not over-rejecting.
  "Stable control mean p implausible" =
    stable_control_degenerate || abs(mean(pvalsStable) - 0.5) < tol_mean + 0.1,
  "Aperiodic control mean p implausible" = abs(mean(pvalsAperiodic) - 0.5) < tol_mean + 0.1,
  "Stable control frac<0.05 far from nominal 5%" =
    stable_control_degenerate || abs(mean(pvalsStable < 0.05) * 100 - 5) < tol_frac + 3,
  "Aperiodic control frac<0.05 far from nominal 5%" = abs(mean(pvalsAperiodic < 0.05) * 100 - 5
)
B_    <- 7.48; C_EA_ <- 0.009; C_PA_ <- 0.004; C_EL_ <- 0.012; U_L_ <- 0.267
U_A_STABLE_ <- 0.73

lpa_step_ <- function(state, u_a) {
  L <- state[1]; P <- state[2]; A <- state[3]
  c(
    B_ * A * exp(-C_EL_ * L - C_EA_ * A),
    L * (1 - U_L_),
    P * exp(-C_PA_ * A) + A * (1 - u_a)
  )
}

# The map is a contraction at the fixed point (see the Jacobian below), so
# direct iteration from an arbitrary interior starting point converges to
# machine precision regardless of where it starts.
state <- c(50, 30, 60)
for (i in 1:5000) state <- lpa_step_(state, U_A_STABLE_)
fixed_point <- state
names(fixed_point) <- c("L", "P", "A")
fixed_point

lpa_jacobian_ <- function(state, u_a) {
  L <- state[1]; P <- state[2]; A <- state[3]
  e <- exp(-C_EL_ * L - C_EA_ * A)
  matrix(c(
    -C_EL_ * B_ * A * e, 0,                  B_ * e - C_EA_ * B_ * A * e,
```

```
    1 - U_L_,            0,                   0,
    0,                   exp(-C_PA_ * A),    -C_PA_ * P * exp(-C_PA_ * A) + (1 - u_a)
  ), nrow = 3, byrow = TRUE)
}

eig <- eigen(lpa_jacobian_(fixed_point, U_A_STABLE_))$values
eig
Mod(eig)

lambda_complex   <- eig[abs(Im(eig)) > 1e-8][1]
rotation_period  <- 2 * pi / abs(Arg(lambda_complex))
contraction_step <- Mod(lambda_complex)
tseq_burnin        <- as.matrix(read.csv("output_burnin/tseq.csv", header = FALSE))[, 1]
DataStable_burnin    <- as.matrix(read.csv("output_burnin/stable_landscapes.csv", header = FALS
DataAperiodic_burnin <- as.matrix(read.csv("output_burnin/aperiodic_landscapes.csv", header = F

n_stable_zero_burnin      <- sum(is_zero_row(DataStable_burnin))
mean_max_aperiodic_burnin <- mean(apply(DataAperiodic_burnin, 1, max))

fdStable_burnin    <- create_fd(DataStable_burnin, tseq_burnin)
fdAperiodic_burnin <- create_fd(DataAperiodic_burnin, tseq_burnin)
fdBoth_burnin      <- create_fd(rbind(DataStable_burnin, DataAperiodic_burnin), tseq_burnin)

pca_burnin     <- pca.fd(fdBoth_burnin, nharm = 2)
pc1_pct_burnin <- pca_burnin$varprop[1] * 100
pc2_pct_burnin <- pca_burnin$varprop[2] * 100

pc1_raw_burnin  <- pca_burnin$scores[, 1]
sign_fix_burnin <- if (mean(pc1_raw_burnin[1:200]) > 0) -1 else 1
PC1_burnin      <- sign_fix_burnin * pc1_raw_burnin
n_aperiodic_pc1_neg_burnin <- sum(PC1_burnin[201:400] < 0)

res_burnin <- run_permtest_suite(
  DataStable_burnin, fdStable_burnin, fdAperiodic_burnin, tseq_burnin,
  input_files = c("output_burnin/tseq.csv", "output_burnin/stable_landscapes.csv", "output_burn
  cache_file  = "output_burnin/results_partB_permtest_cache.rds"
)
test1_burnin <- res_burnin$test1
stable_control_degenerate_burnin <- all(is.na(res_burnin$pvalsStable))

stopifnot(
  "Burn-in n_stable_zero far from sanity-check value" =
    n_stable_zero_burnin == ref_burnin$n_stable_zero,
  "Burn-in mean aperiodic peak far from sanity-check value" =
    abs(mean_max_aperiodic_burnin - ref_burnin$mean_max_aperiodic) < 0.02,
  "Burn-in PC1 far from sanity-check value" = abs(pc1_pct_burnin - ref_burnin$pc1_pct) < 2,
  "Burn-in PC2 far from sanity-check value" = abs(pc2_pct_burnin - ref_burnin$pc2_pct) < 2,
```

```
  "Burn-in aperiodic PC1<0 count far from sanity-check value" =
    abs(n_aperiodic_pc1_neg_burnin - ref_burnin$n_aperiodic_pc1_neg) <= 5,
  "Burn-in Tobs far from sanity-check value" = abs(test1_burnin$Tobs - ref_burnin$test1_Tobs) <
  "Burn-in crit95 far from sanity-check value" = abs(test1_burnin$crit95 - ref_burnin$test1_cri
  "Expected the burn-in stable control to be fully degenerate" = stable_control_degenerate_burn
)
sweep <- read.csv("output/burnin_sweep.csv")
sweep$pct_stable_h1_empty <- sprintf("%.0f%%", 100 * sweep$n_stable_h1_empty / sweep$n_sweep)
knitr::kable(
  data.frame(
    `Burn-in`                   = sweep$burn_in,
    `Stable, H1 empty`          = sprintf("%d / %d (%s)", sweep$n_stable_h1_empty, sweep$n_swe
    `Mean aperiodic peak`       = sprintf("%.4f", sweep$mean_aperiodic_peak),
    check.names = FALSE
  ),
  align = "lrr"
)
sessionInfo()
```